WEEK 4

ACO FOR TSP

```python
import numpy as np
import random

# Distance matrix for 5 cities: A, B, C, D, E
D = np.array([
    [0,  2,  9, 10,  7],
    [2,  0,  6,  4,  3],
    [9,  6,  0,  8,  5],
    [10, 4,  8,  0,  6],
    [7,  3,  5,  6,  0]
], dtype=float)

# Algorithm parameters
m = 10             # Number of ants
N = 100            # Maximum iterations
alpha = 1.0        # Pheromone influence
beta = 2.0         # Distance influence
rho = 0.5          # Pheromone evaporation rate
Q = 100.0          # Pheromone constant
C = 20             # Convergence limit

num_cities = len(D)
city_names = ["A", "B", "C", "D", "E"]

# Step 1: Initialization
pheromone = np.ones((num_cities, num_cities))
best_tour = None
best_distance = float("inf")
no_improvement = 0

random.seed(42)
np.random.seed(42)

# Step 5: Iterate for a maximum of N iterations
for iteration in range(1, N + 1):
```

```
for iteration in range(1, N + 1):

    all_tours = []
    improvement = False

    # Step 2: Construct solutions
    for ant in range(m):

        start = random.randrange(num_cities)
        tour = [start]
        unvisited = set(range(num_cities))
        unvisited.remove(start)

        while unvisited:
            current = tour[-1]
            candidates = sorted(unvisited)

            # Calculate transition probabilities
            weights = np.array([
                (pheromone[current, city] ** alpha) *
                ((1.0 / D[current, city]) ** beta)
                for city in candidates
            ])

            probabilities = weights / weights.sum()

            # Select next city probabilistically
            next_city = int(np.random.choice(
                candidates, p=probabilities
            ))

            tour.append(next_city)
            unvisited.remove(next_city)

        # Return to the starting city
        tour.append(start)

        # Step 3: Evaluate tour
```

```
        # Step 3: Evaluate tour
        length = sum(
            D[tour[i], tour[i + 1]]
            for i in range(num_cities)
        )

        all_tours.append((tour, length))

        # Record the global best tour
        if length < best_distance:
            best_distance = length
            best_tour = tour.copy()
            improvement = True

    # Step 4: Update pheromones
    # Evaporation
    pheromone *= (1 - rho)

    # Deposit pheromones based on tour quality
    for tour, length in all_tours:
        deposit = Q / length

        for i in range(num_cities):
            a = tour[i]
            b = tour[i + 1]
            pheromone[a, b] += deposit
            pheromone[b, a] += deposit

    # Step 5: Check convergence
    if improvement:
        no_improvement = 0
    else:
        no_improvement += 1

    print(
        f"Iteration {iteration}: "
        f"Best distance = {best_distance}"
    )
```

```python
# Step 6: Output best tour
best_route = " -> ".join(
    city_names[city] for city in best_tour
)

print("\n----- FINAL RESULT -----")
print("Best Tour:", best_route)
print("Minimum Distance:", best_distance)
print("Iterations Executed:", iteration)
```

```
Iteration 1: Best distance = 26.0
Iteration 2: Best distance = 26.0
Iteration 3: Best distance = 26.0
Iteration 4: Best distance = 26.0
Iteration 5: Best distance = 26.0
Iteration 6: Best distance = 26.0
Iteration 7: Best distance = 26.0
Iteration 8: Best distance = 26.0
Iteration 9: Best distance = 26.0
Iteration 10: Best distance = 26.0
Iteration 11: Best distance = 26.0
Iteration 12: Best distance = 26.0
Iteration 13: Best distance = 26.0
Iteration 14: Best distance = 26.0
Iteration 15: Best distance = 26.0
Iteration 16: Best distance = 26.0
Iteration 17: Best distance = 26.0
Iteration 18: Best distance = 26.0
Iteration 19: Best distance = 26.0
Iteration 20: Best distance = 26.0
Iteration 21: Best distance = 26.0
Convergence condition reached.

----- FINAL RESULT -----
Best Tour: A -> B -> D -> C -> E -> A
Minimum Distance: 26.0
Iterations Executed: 21
```